

50.039 Deep Learning Final Project

1. Members and Contributions

- 1) Ansh Oswal - 1006265
- 2) Sean Chen Zhi En - 1005122
- 3) Tee Zhi Zhang - 1005136

Ansh was instrumental in researching the latest state-of-the-art models available and in laying out the bulk of the final report. He also architected the transformer model outlined in the report below.

Sean implemented the entire data preprocessing pipeline, developed the custom LSTM and GRU models outlined below, assisted in the research of the latest state-of-the-art models, and developed the scripts to train the state-of-the-art models found.

Zhi Zhang spearheaded the exploratory data analysis (EDA) process, helping us to identify and engineer novel features. He was also in charge of defining the Encoder-Decoder model, and all hyperparameter tuning experiments, which were run on his GPU cluster.

2. Topic & Problem Definition

In the modern travel industry, airfare pricing is highly dynamic, fluctuating based on demand, availability, seasonality, and airline pricing strategies. This variability creates uncertainty for travellers looking to secure the best possible price. Predicting the optimal day to purchase a flight can provide travellers with significant cost savings and enhance their planning experience. Leveraging historical flight price data and machine learning techniques, this project explores the possibility of forecasting future airfare trends to recommend the most cost-effective booking window for one-way flights.

This brings us to our problem statement: How might we help a traveller determine the best time to purchase a flight ticket?

To solve this problem, we developed various models capable of performing Time-Series Forecasting. The expected inputs are flight details such as starting airport, destination airport, flight time, and a sequence of historical flight prices. Based on these inputs the model will predict the price of the flight in the next few days based on which the user can make an informed decision and get the best deal on their flights.

3. Dataset

We utilise the [flight prices dataset from Kaggle](#) (Wong, 2022). The dataset contains real-world flight data sourced from Expedia, covering flights between major U.S. airports over several months in 2022.

3.1. Exploratory Data Analysis

To better understand the data, we first carried out Exploratory Data Analysis. To keep the report concise, we are only including the important findings. However, the entire EDA can be found by opening the *EDA.ipynb* file.

In the exploratory data analysis (EDA) of the flight fare dataset, we observed a total of 38,121 unique route-date combinations. On average, each route-date was searched approximately 5.79 times, with a median of 4 searches. This indicates a slightly skewed distribution where certain routes or dates received significantly more attention than others. The number of searches per route-date ranged from a minimum of 1 to a maximum of 45.

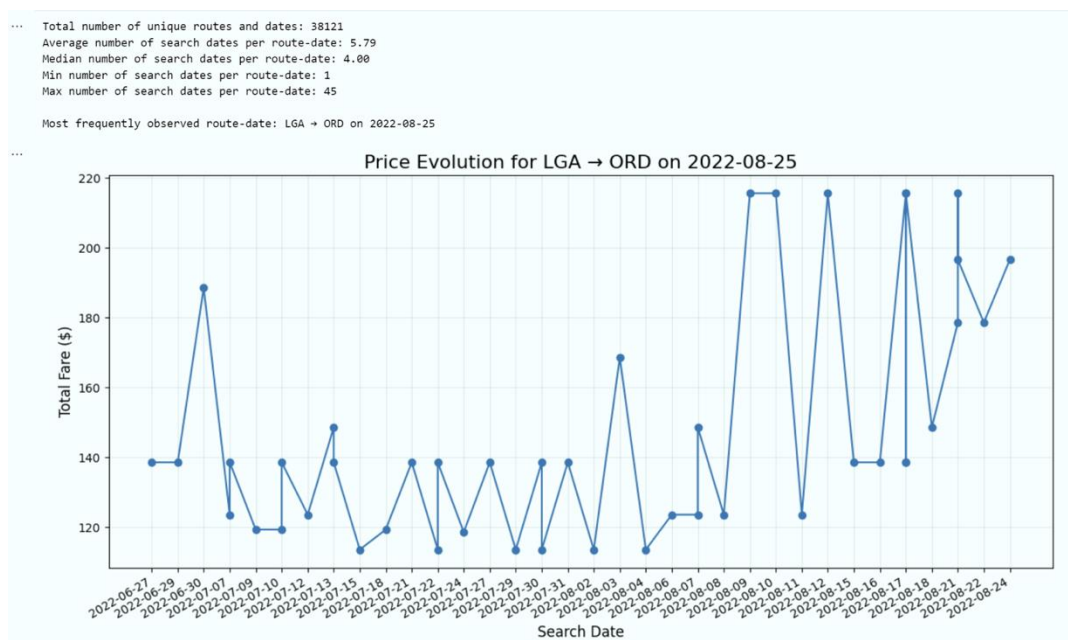


Figure 3.1. Information about search dates, routes, and total fare

The most frequently observed route-date was the flight from LGA to ORD on August 25, 2022. Analyzing the price evolution for this specific route revealed significant fluctuations in fare prices over time, with a general trend of increasing volatility as the departure date approached. Sharp price spikes and dips suggest dynamic pricing influenced by factors such as demand surges and seat availability. This highlights the importance of incorporating temporal patterns into any predictive modeling efforts.

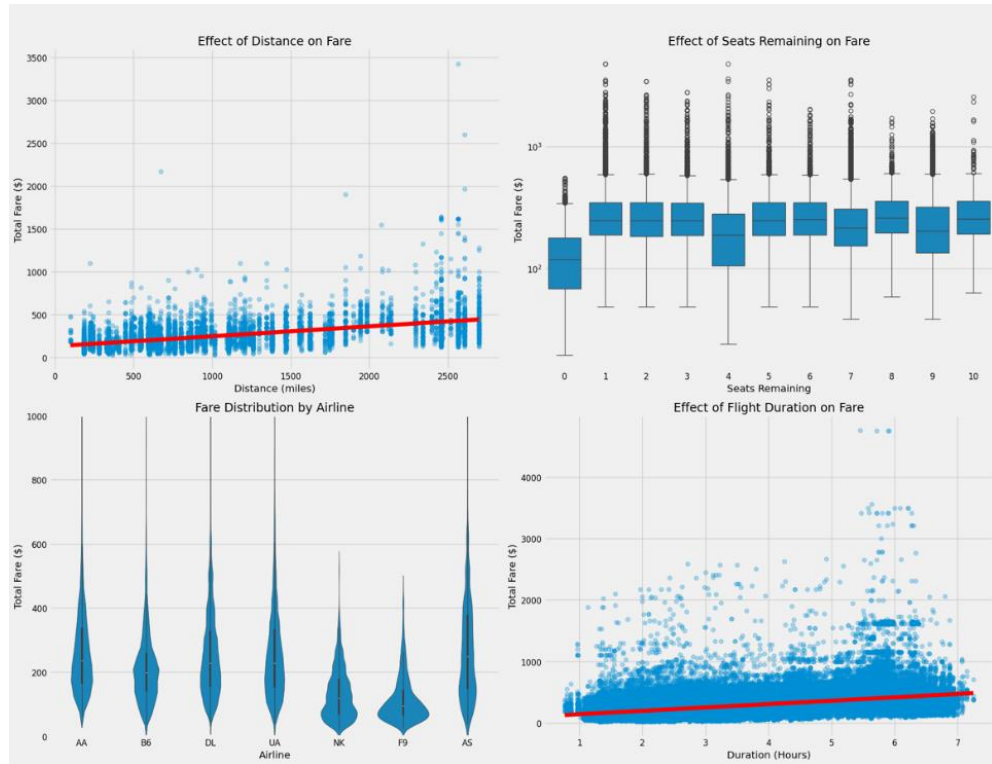


Figure 3.2. Factors Affecting Ticket Prices

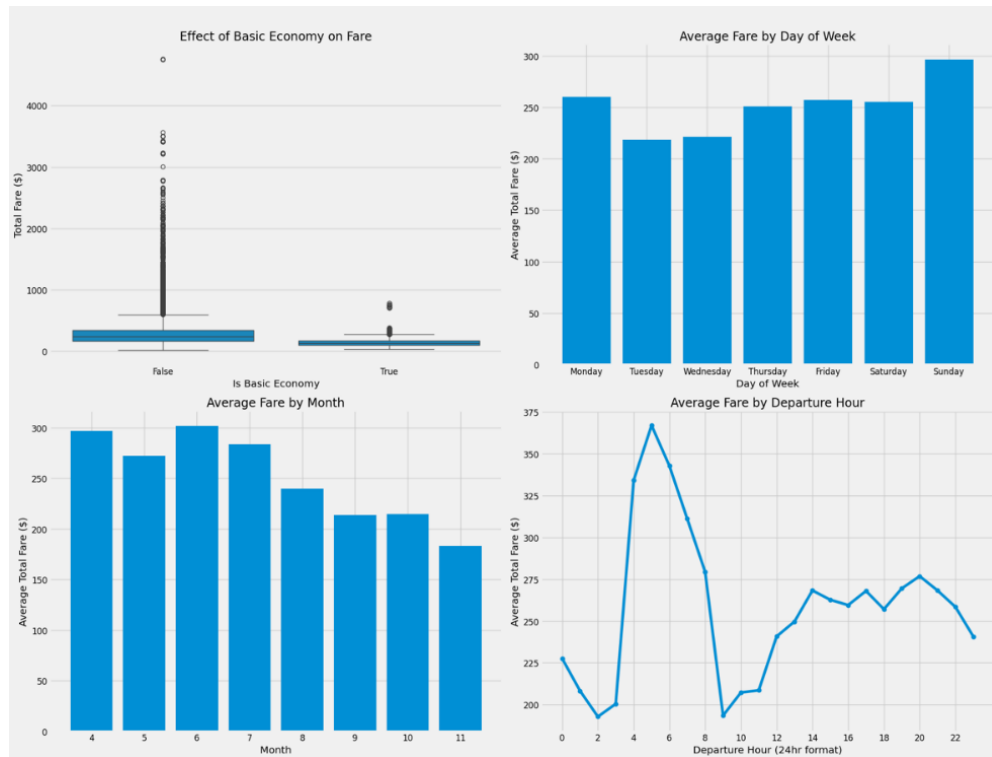


Figure 3.3. Additional Factors Affecting Ticket Prices

The EDA reveals several key factors that impact airline ticket pricing. As expected, both flight distance and duration show a positive correlation with fare, where longer flights generally cost more, though with some variability due to other factors.

Seat availability significantly affects prices. As the number of seats available decreases, the price of the flight increases, reflecting dynamic pricing strategies. Airline choice also matters. For example, budget carriers like Spirit (NK) and Frontier (F9) offer lower fares. Meanwhile, full-service airlines like Delta (DL) and Alaska (AS) are generally more expensive.

Basic economy tickets are consistently cheaper, confirming the role of fare class in pricing. Temporal patterns can also be observed. Fares are highest on Sundays and Mondays, and lowest on Tuesdays and Wednesdays, aligning with travel demand cycles. Seasonally, prices peak from April to July and dip in the fall, reflecting vacation trends.

Finally, departure time influences pricing. Early morning flights of around 5 AM to 6 AM are more expensive. Meanwhile, late-night flights tend to be cheaper. Overall, pricing is shaped by a mixture of operational, airline-specific, and time-based factors.

With all this in mind, we decided to use the following feature variables:

1. *startingAirport*: Three-character IATA airport code for the initial location
2. *destinationAirport*: Three-character IATA airport code for the arrival location
3. *searchDayOfWeek*: *Day of week of the search date*
4. *flightDayOfWeek*: *Day of week of the flight date*
5. *daysBetweenSearchAndFlight*: *Number of days between the search date and flight date*
6. *searchMonth*: *Month of the search date*
7. *flightMonth*: *Month of the flight date*
8. *departureHourUTC*: *Hour that the flight departs in UTC time*
9. *seatsRemaining*: Integer for the number of seats remaining

Meanwhile, our target variable is:

1. *totalFare*: The price of the ticket (in USD) including taxes and other fees

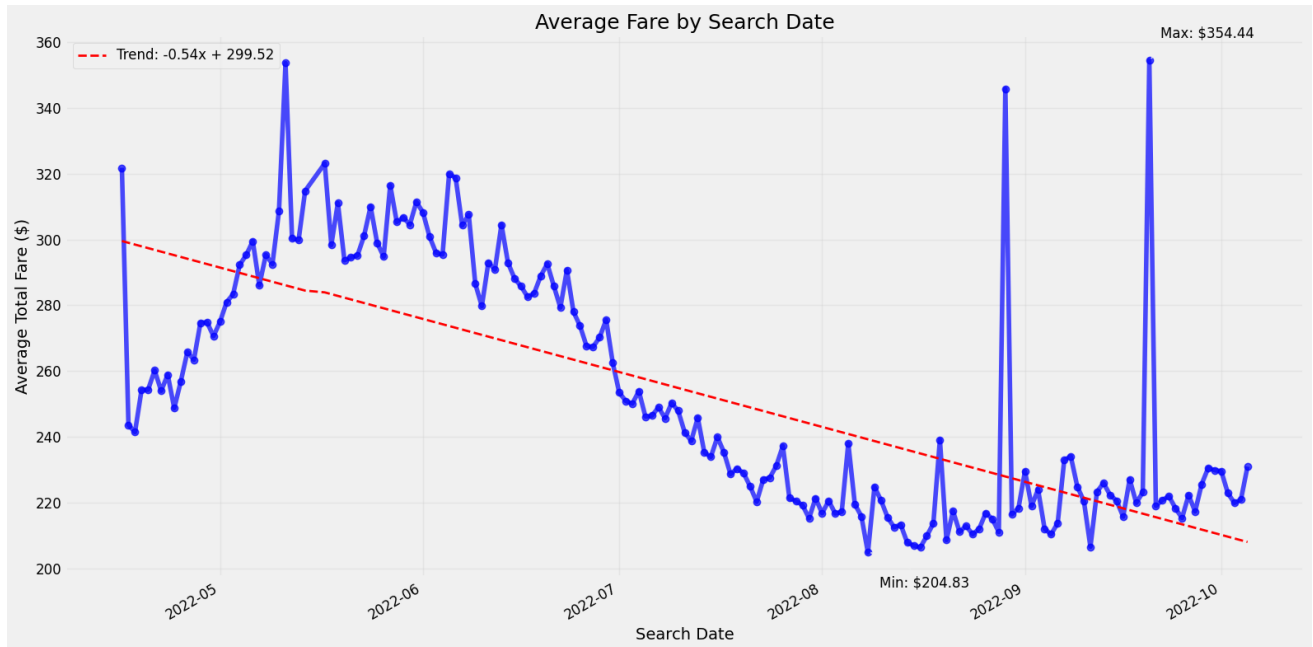


Figure 3.4. Average Fare Prices by Search Date

Figure 3.4 shows the plot of average flight prices on each search date which depicts a strong relationship between the two features. We can observe that prices tend to be higher in the months of May and June, perhaps due to increased demand as a result of the summer holidays, before dropping until September. We also see sharp spikes as the date nears the end of the dataset period, which may be a result of search dates being right before the flight dates.

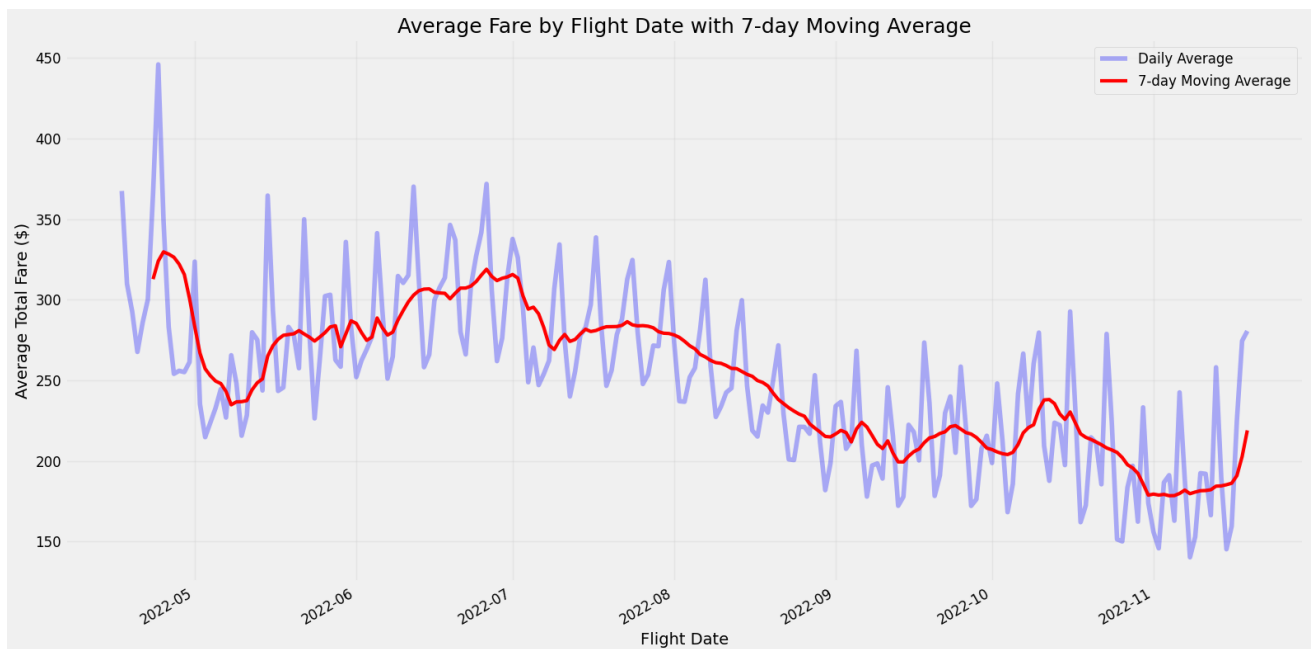


Figure 3.5. Average Fare Prices by Flight Date

Figure 3.5 shows the plot of average flight prices on each date of the flight, which similarly shows interesting patterns in the overall trend and local fluctuations. The periodic spikes in prices can be attributed to weekly price fluctuations depending on the day of the week. The overall trend is similar to the one we observed in Figure 3.4 which was across search dates, suggesting that seasonal factors play an important role in ticket pricing as well. Figure 3.5

3.2. Data Preprocessing

We performed the following preprocessing steps.

3.2.1. Sub-sampling

Firstly, since the original dataset was more than 30 GB, we decided to focus only on direct flights (*isNonStop = True*). Also, we took a random sample of 10% of the filtered dataset. Finally, we removed the following columns to save space:

1. *travelDuration*: The travel duration in hours and minutes
2. *segmentsDepartureTimeRaw*: String containing the departure time in ISO 8601 format
3. *segmentsArrivalTimeRaw*: String cotaining the arrival time in ISO 8601 format
4. *segmentsAirlineName*: String containing the name of the airline that services each leg of the trip
5. *segmentsDistance*: String containing the distance traveled (in miles) for each leg of the trip
6. *segmentsArrivalAirportCode*: String containing the IATA airport code for the arrival location for each leg of the trip
7. *segmentsDepartureAirportCode*: String containing the IATA airport code for the departure location for each leg of the trip

These columns could be removed because each column could be represented by another column in the dataset.

3.2.2. Grouping and Aggregation

Next, we grouped all flights by the following combination of features:

10. *startingAirport*: Three-character IATA airport code for the initial location
11. *destinationAirport*: Three-character IATA airport code for the arrival location
12. *flightDate*: The date (YYYY-MM-DD) of the flight
13. *searchDate*: The date (YYYY-MM-DD) on which the flight entry was taken from Expedia

If there are multiple rows with the same combination of features, we take the average of the following columns:

2. *baseFare*: The price of the ticket (in USD)
3. *totalFare*: The price of the ticket (in USD) including taxes and other fees
4. *seatsRemaining*: Integer for the number of seats remaining

3.2.3. Feature engineering

Then, we engineered the following features:

1. *searchDayOfWeek*: *Day of week of the search date*
2. *flightDayOfWeek*: *Day of week of the flight date*
3. *daysBetweenSearchAndFlight*: *Number of days between the search date and flight date*
4. *searchMonth*: *Month of the search date*
5. *flightMonth*: *Month of the flight date*
6. *departureHourUTC*: *Hour that the flight departs in UTC time*

3.3. Train/Validation/Test split

$$sequenceLength = inputSequence + outputSequence$$

$$inputSequence = \text{Number of historical values}$$

$$outputSequence = \text{Number of values to predict}$$

Groups with fewer than *sequenceLength* rows were discarded. Then, the remaining rows were split into a training, validation and test set. Since our dataset is large, we allocated 60% for training, 20% for validation, and 20% for testing.

3.4. Sliding window

For each flight group outlined in Section 3.2.2, if the group has at least *sequenceLength* rows, we sort the group by *searchDate* in ascending order. Then, we slide a fixed-length frame of size *sequenceLength* across the sorted group, advancing one row at a time. For each window, we add a new sample to our list of training samples. For *sequenceLength* = 30, this increased our total training sample size by more than 10 times. In particular, our training samples increased from 14,381 to 163,949.

3.5. Encoding

All numerical features were normalised to have a mean of 0 and a standard deviation of 1.

Meanwhile, we performed cyclical encoding on temporal features. Specifically, we mapped each temporal feature onto a unit circle using sine and cosine functions. This helps the model to understand that the last value of a temporal feature is close to the first value. For example, cyclical encoding helps the model to understand that Sunday is close to Monday.

Finally, we performed one-hot encoding on categorical features such as the starting and destination airport.

Assuming $inputSequence = 15$, the number of samples, features, and sequence length after preprocessing is shown in Table 3.1.

	Sample size	Number of features	Sequence length
Train	163,949	15	15
Validation	20,132		
Test	20,007		

Table 3.1. Sample size, number of features, and sequence length for each dataset

4. Model Architectures and Experimentation

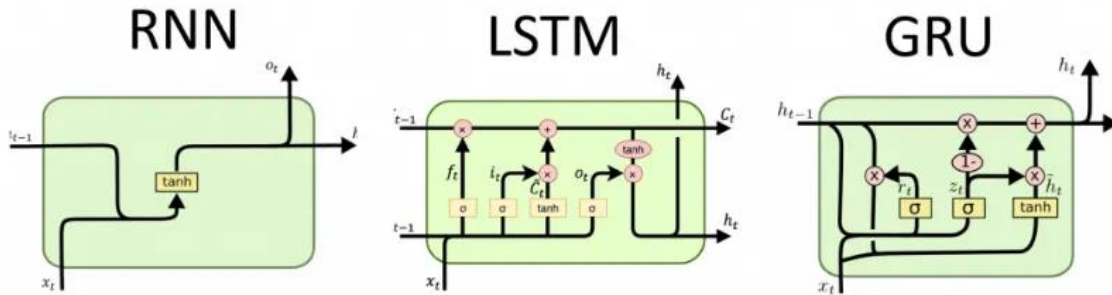


Figure 4.1. Comparison of RNN-based Units (DProgrammer Org, 2019)

For this problem we decided to start experimenting with three models that we thought would be suitable for Time Series Forecasting.

4.1. GRU

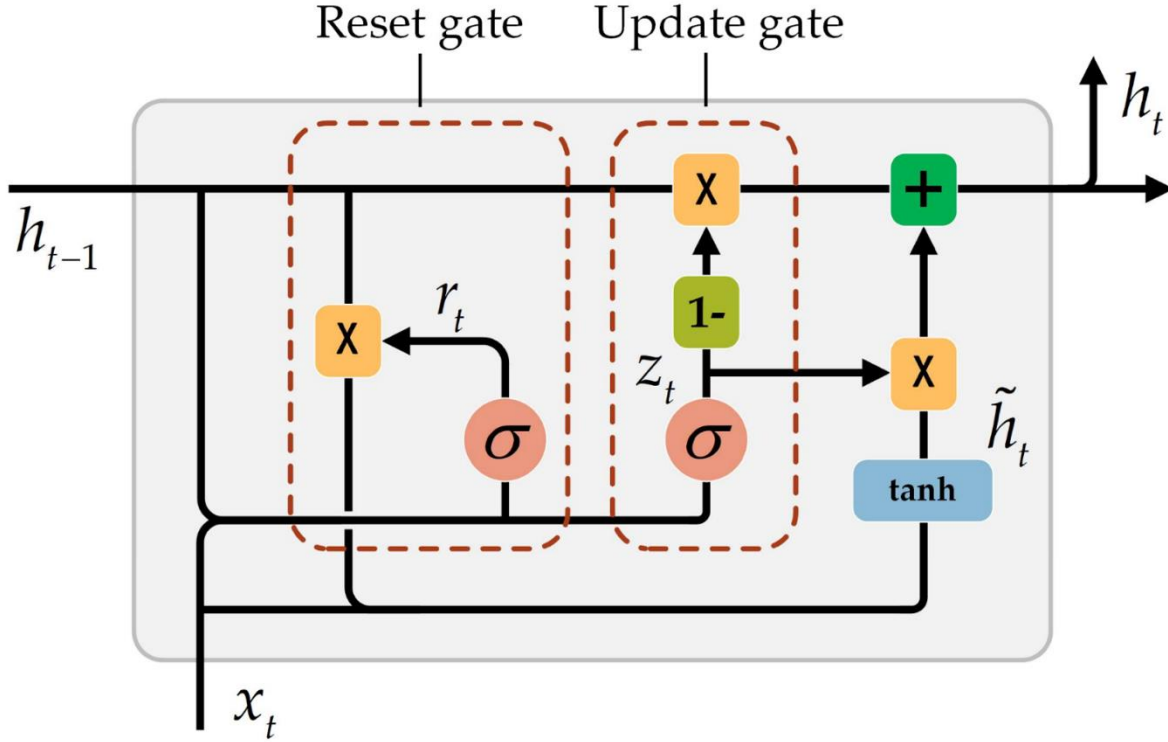


Figure 4.2. GRU Architecture (Li et al., 2020)

The Gated Recurrent Unit (GRU) model was designed to efficiently capture temporal dependencies within flight price sequences. GRUs simplify the structure of traditional RNNs by introducing two gates: an update gate and a reset gate. These gates control the flow of information and help mitigate the vanishing gradient problem. In this project, we experimented with both a basic and a deep GRU architecture.

The basic GRU model consists of a single GRU layer followed by a fully connected linear layer to map the hidden state of the final time step to the output price. For more complex temporal patterns, I designed a deeper variant with multiple GRU layers and a stacked fully connected block, where the output from the last time step is passed through two dense layers with ReLU activations and dropout for regularization before producing the final price prediction. This combination allows the model to balance sequence learning with non-linear transformations, improving its capacity to generalize.

4.2. LSTM

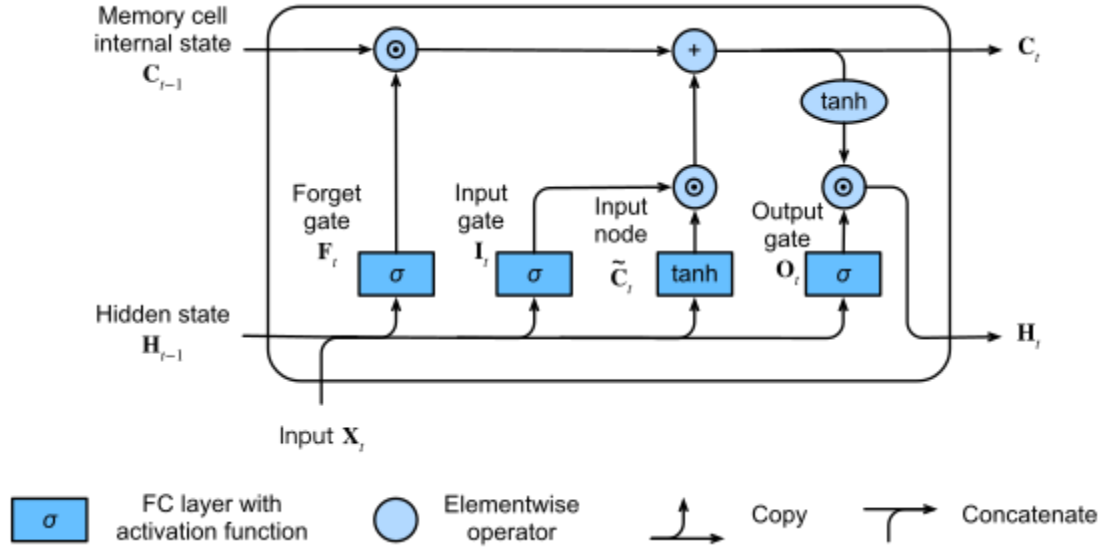


Figure 4.3. LSTM Architecture (Calzone, 2022)

Long Short-Term Memory (LSTM) networks are another variant of recurrent neural networks, widely used for time series forecasting tasks due to their ability to learn long-term dependencies. LSTMs introduce a more complex internal architecture featuring three gates. The three gates are: input, forget, and output gates. These gates regulate the flow of information through the network. This enables the model to selectively retain relevant patterns from past sequences while discarding irrelevant information, making it particularly well-suited for learning the fluctuating nature of flight prices over time. For this project, the LSTM-based model was designed to process both the sequential price data and associated flight attributes, allowing it to predict future price trends based on historical behaviour.

The basic model uses a single LSTM layer to process historical price sequences, with the hidden state from the last time step fed into a fully connected layer to predict future prices. The deep LSTM model extends this design by stacking multiple LSTM layers, adding dropout between them, and using a multi-layer dense block with ReLU activations for the final prediction. This layered setup enables the model to capture complex sequential relationships while reducing the risk of overfitting.

4.3. LSTM Encoder-Decoder

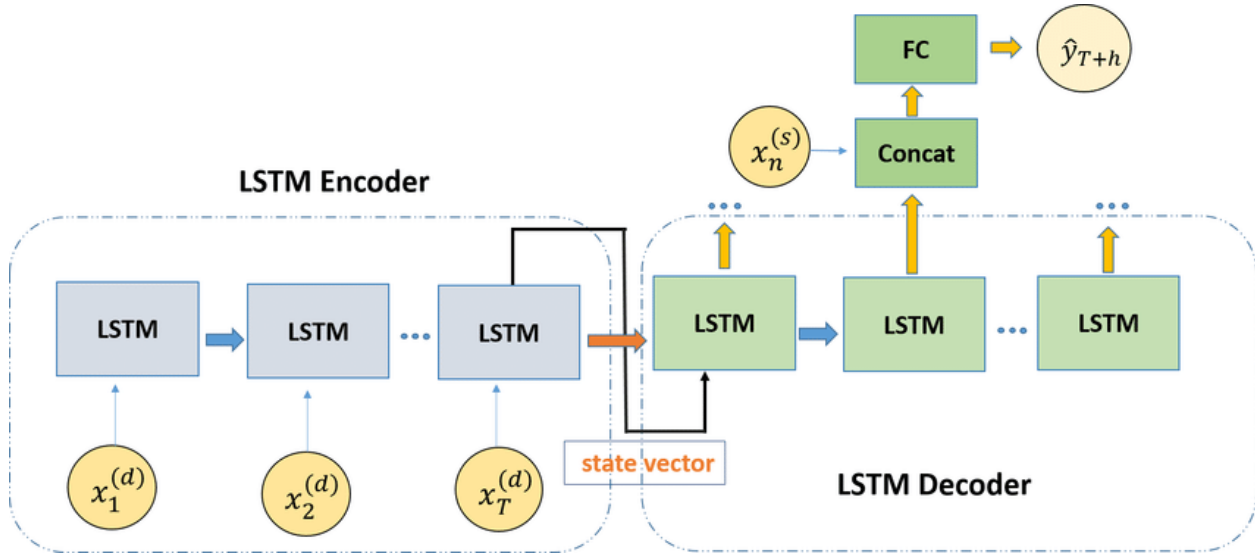


Figure 4.4. LSTM Encoder-Decoder Architecture (Jacoby et al., 2021)

The LSTM Encoder-Decoder represents a sequence-to-sequence architecture specifically adapted for time series forecasting tasks. This model consists of two primary components: an encoder that processes and compresses the input sequence into a context representation, and a decoder that transforms this representation into the target prediction.

The encoder is implemented as a multi-layered LSTM network that processes the entire input time series. For each input sequence X of shape (batch size, sequence length, input features), the encoder processes the temporal patterns and relationships within the data. It compresses this information into two state vectors: Hidden state (h): Representing the short-term memory. Cell state (c): Capturing long-term dependencies. These states collectively form a fixed-dimensional representation of the variable-length input sequence, effectively compressing the relevant temporal information.

The decoder consists of another multi-layered LSTM network that receives the final hidden and cell states from the encoder as its initial states, takes a minimal initial input, leverages the encoded representation to generate the prediction.

A final fully connected layer transforms the decoder's output into the desired prediction format, mapping from the hidden dimension to the output dimension.

4.4. Transformer

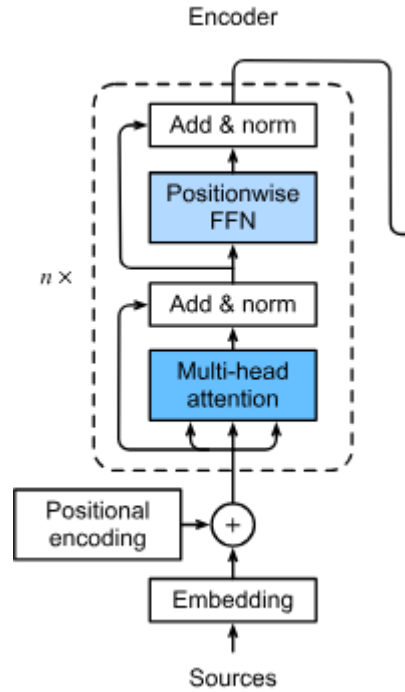


Figure 4.5. Transformer Encoder Architecture (Dive Deep into Deep Learning, n.d.)

The Transformer model employed in this project was inspired by the original architecture introduced for natural language processing but adapted for time series forecasting. Unlike GRU and LSTM models, the Transformer is entirely non-recurrent and relies on self-attention mechanisms to learn the relationships between all positions in a sequence simultaneously.

In this implementation, each input sequence is first passed through a linear projection layer to match the model's hidden dimensionality. To help the model retain information about the position of each element in the sequence, I incorporated sinusoidal positional encoding, which is added to the input embeddings. The core of the model is a stack of Transformer encoder layers, each containing multi-head self-attention and feedforward sublayers, allowing the model to dynamically focus on relevant past data points. The output from the final encoder layer is reduced to the last time step and passed through a linear layer to produce the predicted price.

5. State of the Art Models

5.1. TimesNet

Introduced in 2023, [TimesNet](#) (Wu et al., 2023) is a sophisticated deep learning architecture designed for multivariate time series analysis. TimesNet leverages frequency-domain analysis and an efficient representation learning mechanism to capture temporal dependencies and periodic patterns inherent in multivariate time series data.

The architecture consists of two key components: a frequency-domain embedding module and a representation learning module. Initially, the model performs a discrete Fourier transform (DFT) on the input time series, translating temporal information into frequency-domain features. Subsequently, TimesNet applies learnable frequency-domain embeddings that selectively highlight essential periodic patterns while suppressing noise or irrelevant frequency components.

Following frequency-domain embedding, the model incorporates a stack of convolutional neural network (CNN) layers combined with residual connections to capture intricate temporal dependencies across various scales. Additionally, adaptive normalisation techniques are applied, further stabilising training and enhancing generalisation capabilities.

By combining frequency-aware embeddings with convolution-based learning, TimesNet effectively identifies and extracts critical temporal patterns within complex multivariate time series. Currently, TimesNet is one of the best model architectures for forecasting time series, according to this [repository of time-series evaluations](#).

5.2. DLinear

Introduced in 2022, [DLinear](#) (Zeng et al., 2023) is a minimalist deep learning approach tailored specifically for multivariate time series forecasting. Unlike more complex methods, DLinear leverages a purely linear framework to efficiently model the temporal dynamics of a dataset.

The core concept behind DLinear involves decomposing input time series data into a trend and a seasonal component using moving averages. Each of these decomposed sequences is then independently processed using simple linear layers. The linear layers map historical observations directly to future predictions, effectively capturing the fundamental temporal structures in the data without reliance on sophisticated nonlinearities or recurrent architectures.

After processing the trend and seasonal components separately through parallel linear layers, the outputs are combined to yield the final prediction. This explicit decomposition strategy allows DLinear to directly model patterns such as long-term trends and repeating seasonal behaviours, significantly enhancing forecasting accuracy and interpretability.

While DLinear has since been surpassed by models such as TimesNet, its linear architecture makes it significantly less computationally expensive than TimesNet.

6. Methodology

To optimize the performance of our models, hyperparameter tuning was a key aspect of our project. We first tuned hyperparameters that were deemed more “generic”, such as learning rate, batch size, input and output sequence lengths, the hidden dimensions of, and number of fully connected layer(s) in our predictor at the last layer of the model. These hyperparameters were considered as more “generic” in the sense that they apply across all types of model architectures.

6.1. LSTM/GRU Models Grid Search

We first tune these hyperparameters on both a basic LSTM and GRU model first, to reduce the time and computational load required for tuning the bulk of hyperparameters.

Our first set of grid-search experiments were across different values of learning rates, number of fully connected layers in our predictor, along with the hidden dimensions of these fully connected layers, and the hidden dimension of our LSTM/GRU models. Here are the top 5 best performing combinations of hyperparameters from the grid search based on the mean squared error on our validation test set:

model_type	learning_rate	num_layers	fc_hidden_sizes	model_hidden_size	sequence_length	batch_size	val_mse	val_mae
LSTM	0.0005	1	64	128	30	32	8260.499872	50.974907
GRU	0.0005	1	128	128	30	32	8270.565345	51.882754
LSTM	0.0010	1	128	64	30	32	8280.286554	51.512347
GRU	0.0005	2	256_128	128	30	32	8292.049970	51.224804
LSTM	0.0005	3	512_256_128	64	30	32	8299.395556	51.256519

Table 6.1.

To more easily determine the optimal configuration of hyperparameters, we grouped the results according to each hyperparameter value and model type, before taking the average of the mean absolute error of all grouped rows to for better comparison:

6.1.1. Learning Rate

learning_rate	model_type	val_mae	val_mse
0.0005	LSTM	52.408399	8501.642955
0.0010	LSTM	52.456292	8487.081720
0.0005	GRU	52.540383	8516.080330
0.0010	GRU	52.714933	8539.028380
0.0500	GRU	73.520431	13993.019500
0.1000	GRU	78.040754	15500.355216
0.0500	LSTM	80.901124	16713.244522
0.1000	LSTM	87.218518	18725.188929

Table 6.2.

6.1.2. Model Hidden Size

model_hidden_size	model_type	val_mae	val_mse
64	GRU	62.074649	11012.727671
16	GRU	62.436732	10878.107191
16	LSTM	62.609787	11182.972388
32	GRU	62.642135	11290.779410
128	GRU	65.698954	12264.771464
32	LSTM	66.195751	12461.450701
64	LSTM	68.850143	13398.082677
128	LSTM	69.884128	13776.440330

Table 6.3.

6.1.3. Number of FC Layers

num_layers	model_type	val_mae	val_mse
1	GRU	57.948879	9601.851015
1	LSTM	62.290979	11107.298782
3	LSTM	65.266811	12194.556399
3	GRU	65.727354	12242.316706
2	GRU	66.434983	12405.405776
2	LSTM	72.721520	14691.978680

Table 6.4.

6.1.4. FC Hidden Dimensions

fc_hidden_sizes	model_type	val_mae	val_mse
128	GRU	56.877263	9283.849772
16	GRU	57.448535	9424.543490
64	GRU	57.756218	9531.485385
64	LSTM	59.057289	10121.062460
32	GRU	59.713501	10167.525414
32	LSTM	62.845611	11232.162675

256_128_64	LSTM	62.978792	11502.576552
128	LSTM	63.123071	11285.491049
512_256_128	LSTM	63.332453	11540.511159
64_32_16	GRU	63.398937	11264.500433
16	LSTM	64.137947	11790.478945
128_64_32	GRU	64.315555	11858.907205
256_128_64	GRU	64.361982	11829.242473
256_128	GRU	65.210483	12113.000180
128_64_32	LSTM	65.637956	12363.889161
128_64	GRU	65.841902	12227.786222
32_16	GRU	67.183083	12454.579424
64_32	GRU	67.504464	12826.257280
64_32_16	LSTM	69.601631	13534.760035
512_256_128	GRU	69.811824	13661.756711
256_128	LSTM	69.822106	13720.648897
128_64	LSTM	71.138995	14195.389993
64_32	LSTM	73.749430	15007.493120
32_16	LSTM	76.175547	15844.382712

Table 6.5.

Based on the above experiments, we can conclude that the optimal learning rate to use for both LSTM and GRU is 0.0005 or 0.001, and the hidden dimension should be 16. Interestingly, LSTM works better with one fully connected layer with hidden dimension of 16 whereas GRU works better with one fully connected layer with hidden dimension of 128. We also observe that LSTM and GRU based models perform similarly.

For our second set of experiments for LSTM and GRU models, we varied the batch size, sequence length, and with lower values for the hidden dimensions of the models, while fixing the learning rate at 0.001 and using one fully connected layer with hidden dimension of 64. In our experiments, sequence length means the total length of the history input sequence and the predicted output sequence, with both sequences being the same length (i.e. sequence length of 30 means input history sequence of length 15 and predicted output sequence being 15). Below are the top 5 best performing hyperparameter combinations based on the mean squared error achieved on our validation test set:

model_t ype	batch_size	sequence_length	model_hidd en_size	learning rate	num_layers	fc_hidden_s izes	val_mse	val_mae
LSTM	64	16	32	0.001	1	64	7876.21127 1	53.316108
LSTM	128	16	32	0.001	1	64	7883.09919 3	52.968368
LSTM	32	16	32	0.001	1	64	7926.70644 4	53.206084
GRU	128	16	32	0.001	1	64	7947.03077 6	53.304042
LSTM	128	16	16	0.001	1	64	7985.27927 6	53.498982

Table 6.6.

The aggregated results for each hyperparameter value are as follows:

6.1.5. Sequence Length with Model Hidden Size

sequence_length	model_hidden_size	model_type	val_mae	val_mse
30	32	LSTM	51.956867	8420.464997
30	32	GRU	52.446200	8489.570305
30	16	LSTM	52.711403	8543.195937
16	32	LSTM	53.163520	7895.338969
30	16	GRU	53.198189	8628.477836
16	32	GRU	53.364954	7989.796352
8	32	LSTM	53.660907	8189.525279
16	16	LSTM	53.692721	8005.788263
8	32	GRU	53.712214	8213.885935
8	16	LSTM	53.952855	8290.993268
8	16	GRU	53.984554	8320.046263
16	16	GRU	54.045829	8082.406358
8	4	LSTM	54.626860	8467.069003
8	4	GRU	54.661644	8482.539056
16	4	LSTM	54.801065	8240.639992
16	4	GRU	54.917588	8264.613579
30	4	LSTM	54.954173	8974.543780

30	4	GRU	55.006888	9053.959359
2	32	LSTM	60.766996	10550.748554
2	32	GRU	61.067908	10593.305442
2	16	LSTM	61.541857	10768.038713
2	16	GRU	61.986956	10929.624403
2	4	GRU	62.665996	11155.015726
2	4	LSTM	62.767126	11179.364267

Table 6.7.

6.1.6. Batch Size

batch_size	val_mae	val_mse
64	55.782449	8988.135985
128	55.804064	8993.362441
32	55.870396	8984.620528

Table 6.8.

From the above results, we observe that for both LSTM and GRU models, the optimal sequence length and model hidden size are 30 and 32 respectively. The batch size parameter does not have a significant impact on the results for both models, and we stick to a value of 32 for faster processing.

6.1.7. Flight-Price as only input feature

Another area of experimentation was the input features used in our models. In all our experiments thus far, we have been passing in all 10 of the available input features into the model, which we hypothesised could be too much information for the models to process. Hence, we ran a set of experiments by removing all other features except for the flight prices, which is our target variable, effectively reducing our multivariate model into a simpler univariate one. We again varied the sequence lengths, model hidden dimensions and batch sizes in this set of experiments. Here are the top 5 results based on the validation mean squared error:

model_type	batch_size	sequence_length	model_hidden_size	learning_rate	num_layers	fc_hidden_sizes	val_mse	val_mae
GRU	64	16	16	0.001	1	64	8577.727733	56.327462
LSTM	64	16	32	0.001	1	64	8579.920739	56.070550

LSTM	64	16	16	0.001	1	64	8584.11 1481	56.374306
LSTM	32	16	16	0.001	1	64	8584.19 9039	56.162815
LSTM	32	16	32	0.001	1	64	8585.77 9748	56.470700

Table 6.9.

From these results, it is clear that removing all the other input features does not improve our models' performance, and therefore we stick to using all input features in the subsequent grid search experiments for other base models.

The full grid search experiment results can be found in the *hyperparam_results_lstm_gru_analysis.ipynb* notebook.

6.2. Encoder-Decoder LSTM Models Grid Search

Next, we move on to a slightly more complex model architecture: using two LSTM models, one being the encoder and the other the decoder in an autoencoder model. Here, we vary the sequence lengths, hidden dimensions inside each LSTM model, and the number of LSTM layers used inside the encoder and decoder, keeping the other hyperparameters constant based on the optimal values obtained in our first few grid search experiments. Here are the top 5 hyperparameter combinations based on the validation mean squared error for the encoder-decoder model:

model_type	batch_size	sequence_length	model_hidden_size	learning_rate	num_layers	fc_hidden_sizes	val_mse	val_mae
EncoderDecoder	32	16	32	0.005	1	64	7972.115015	53.477589
EncoderDecoder	32	16	16	0.005	1	64	8007.533197	53.538555
EncoderDecoder	32	16	16	0.005	5	64	8076.612346	54.281078
EncoderDecoder	32	16	32	0.005	5	64	8091.594463	53.815736
EncoderDecoder	32	16	4	0.005	5	64	8234.557545	54.895260
EncoderDecoder	32	16	4	0.005	1	64	8239.241182	54.763806

Table 6.10.

Next, we again group our results according to each hyperparameter value below:

6.2.1. Sequence Length with Model Hidden Size

model_hidden_size	sequence_length	val_mae	val_mse
32	16	53.646663	8031.854739
16	16	53.909817	8042.072771
4	16	54.829533	8236.899364
32	2	58.940451	10120.426151
16	2	60.406966	10490.887584
4	2	62.912150	11199.196535
32	30	75.701773	15759.328372
16	30	75.988848	15817.009209
4	30	76.882695	16093.239547

Table 6.11.

6.2.2. Num of Layers of Encoders and Decoders

num_layers	val_mae	val_mse
1	56.173422	9169.355877
5	71.208555	13895.291739

Table 6.12.

From the above results, we see that Encoder-Decoder with LSTM models perform best with a sequence length of 16, with a hidden dimension of 32 in just one LSTM layer in both the encoder and decoder. Having more LSTM layers does not necessarily improve the overall model performance.

The full grid search experiment results can be found in the *hyperparam_results_encoderdecoder_analysis.ipynb* notebook.

6.3. Transformer Models Grid Search

Lastly, we move on to performing hyperparameter tuning for transformers-based models. We tune the following parameters: number of transformer encoder layers, the hidden dimension of each transformer encoder layer, number of attention heads used, and the dropout rate for each transformer encoder layer. We use a single fully connected layer after the transformer encoder

layer as our predictor layer and keep other hyperparameters fixed based on the initial set of experiments. Here are the best 5 hyperparameter combinations for our transformer-based models based on validation mean squared error:

transformer_num_layer	transformer_hidden_size	transformer_n_head	transformer_dropout	learning_rate	model_hidden_size	sequence_length	batch_size	val_mse	val_mae
2	16	4	0.0	0.005	32	30	32	8616.604335	53.474585
2	16	2	0.0	0.005	32	30	32	8630.799396	53.052134
2	16	1	0.0	0.005	32	30	32	8664.985853	52.862627
1	32	8	0.0	0.005	32	30	32	8773.263330	54.640006
1	16	8	0.0	0.005	32	30	32	8791.146553	54.219069

Table 6.13.

6.3.1. Number of Transformer Encoder Layers

transformer_num_layer	val_mae	val_mse
1	57.723643	9567.380099
2	59.267043	9786.700908
4	95.312727	21479.534985

Table 6.14.

6.3.2. Hidden Dimension of Transformer Encoder Layers

transformer_hidden_size	val_mae	val_mse
16	64.292356	11756.847624
32	71.782701	14023.689164
64	73.436911	14328.203472
128	73.559249	14336.081063

Table 6.15.

6.3.3. Number of Attention Heads

transformer_n_head	val_mae	val_mse
4	69.554293	13267.560657
8	70.146024	13478.742966

2	70.553690	13551.160324
1	72.817209	14147.357375

Table 6.16.

6.3.4. Dropout Rate

transformer_dropout	val_mae	val_mse
0.1	69.202854	13161.121063
0.0	70.666244	13723.069221
0.2	70.749555	13569.870638
0.3	72.452564	13990.760401

Table 6.17.

Based on the above results, we see that the optimal number of transformer encoder layers to use is either 1 or 2, with each layer having a hidden dimension of 16. The number of attention heads and the dropout rate do not impact our model performance significantly, but we see that using 4 attention heads and 0.1 dropout rate yielded slightly better performance across all experiments.

The full grid search experiment results can be found in the *hyperparam_results_transformers_analysis.ipynb* notebook.

7. Comparison of Best Models with SOTA Models

We save the weights of the best models for each of the architectures obtained from the grid search experiments above in the *best_models_weights* folder in the repository.

Comparing the best models with the SOTA models which were ran with suggested default configurations on the same training and validation datasets, we can benchmark the models' performance:

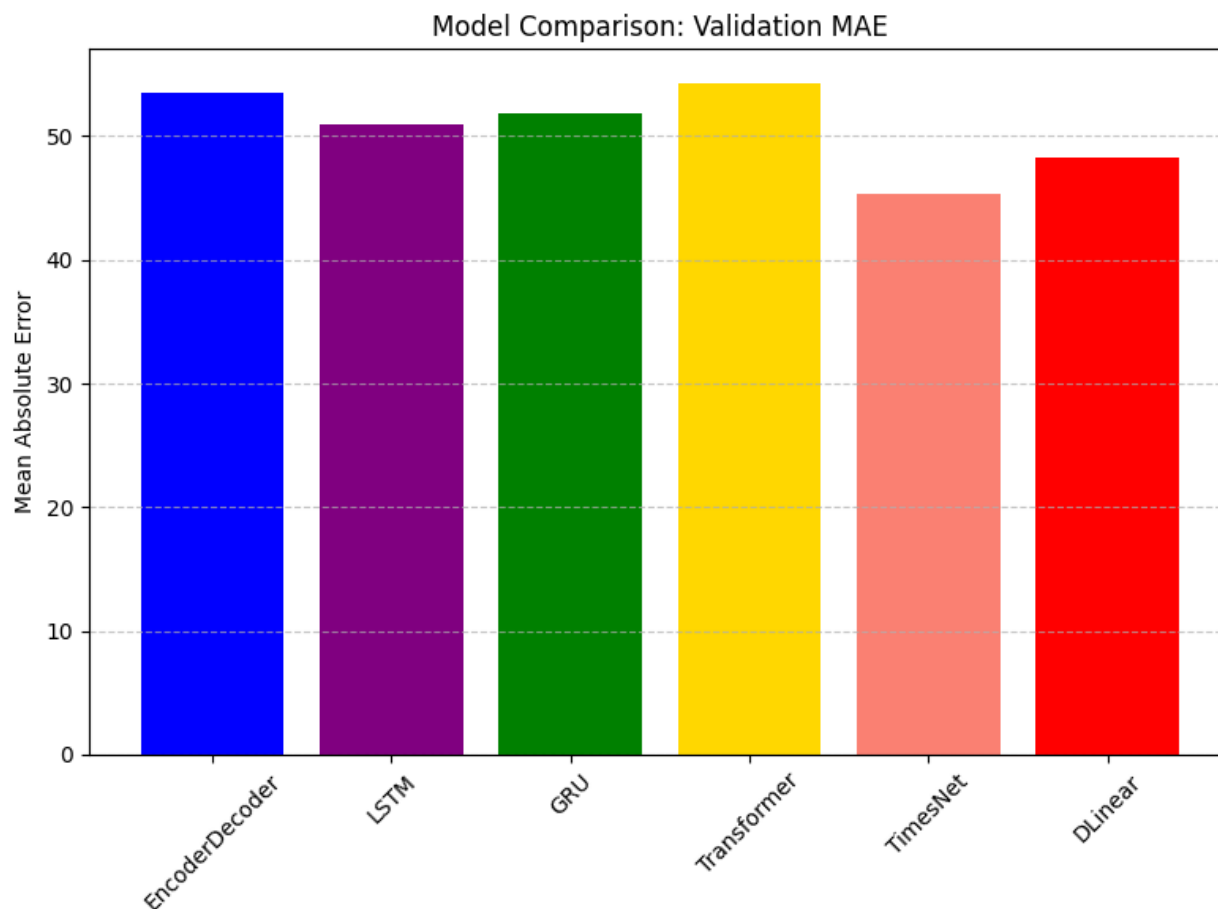


Figure 7.1 Overall Comparison across All Model Architectures Tested

As per Figure 7.1, we see that TimesNet outperforms all other architectures, demonstrating superior accuracy for this sequence modeling task. DLinear, despite its simplicity, performs better than all traditional recurrent and attention-based models, making it an efficient choice. Traditional recurrent models like LSTM and GRU remain solid performers, though slightly outpaced by newer models. Surprisingly, Transformer underperforms, suggesting it may not be the best fit here unless optimized further or provided with more data. The LSTM Encoder-Decoder model shows middling performance and might require architecture-specific tuning to perform competitively.

8. Model Evaluation

8.1. TimesNet

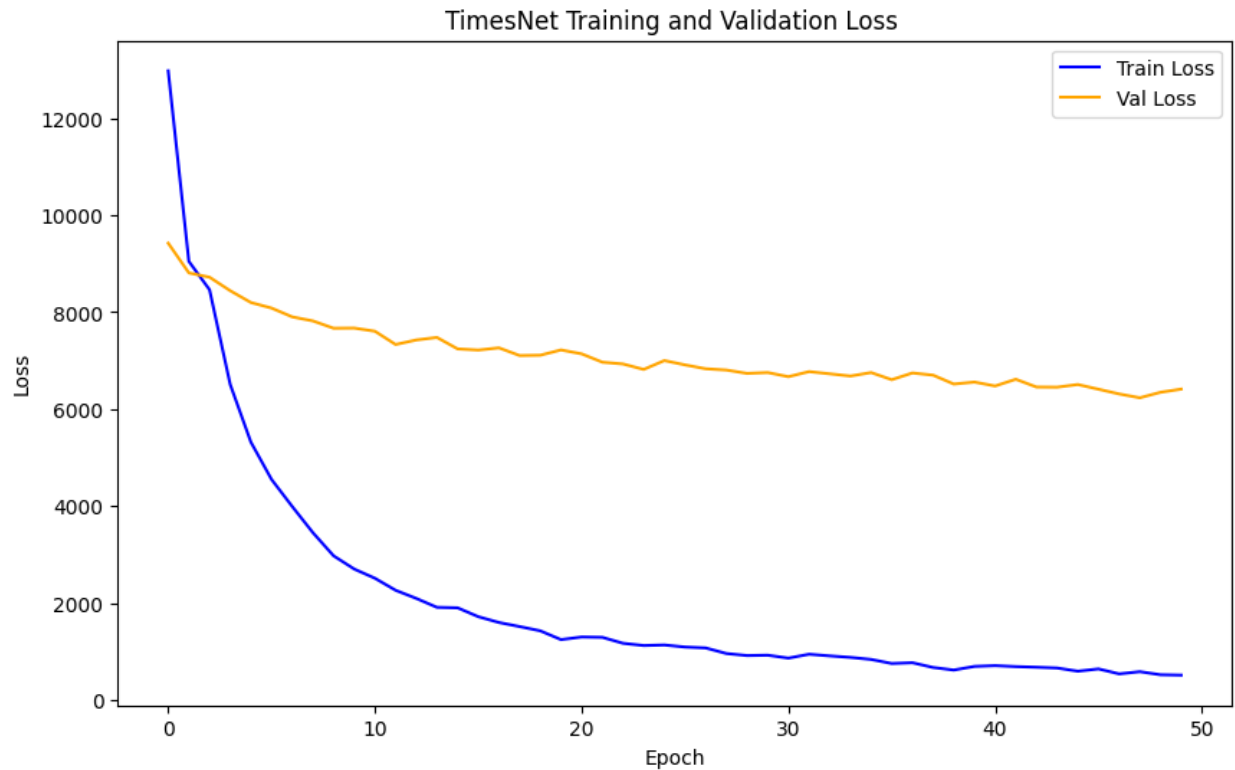


Figure 8.1. Training and Validation Loss of the Best TimesNet model

The training loss (blue) exhibits a sharp and consistent decline, indicating that the model is learning effectively and fitting the training data well. Similarly, the validation loss (orange) follows a steady downward trend, albeit at a more moderate pace, reflecting improved generalization to unseen data. Both curves converge around the 6400 mark, which represents the lowest loss achieved across all models tested so far. The smooth and stable nature of the loss curves further suggests that the learning rate has been appropriately tuned, allowing the model to avoid erratic gradient updates and ensuring a balanced and efficient convergence process.

8.2. DLinear

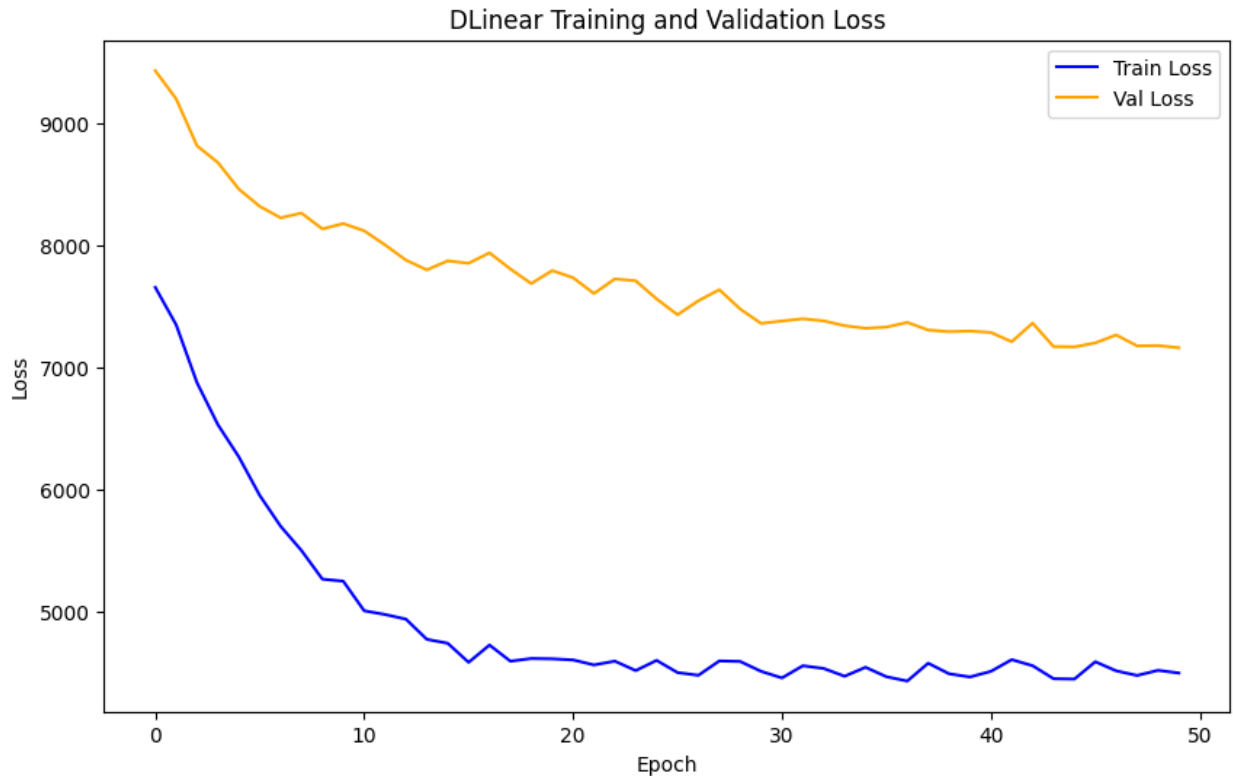


Figure 8.2. Training and Validation Loss of the Best DLinear model

DLinear also demonstrated consistent and reliable results throughout training. Both the training and validation losses show a steady downward trend, highlighting the model's ability to learn and generalize effectively. The validation loss appears to converge around the 7000 mark. While the absolute loss values are slightly higher compared to TimesNet, DLinear ultimately outperformed all our models, narrowly surpassing LSTM, which had previously delivered the best performance in our experiments.

It is also worth noting that the loss curves for DLinear are noticeably less smooth than those observed for TimesNet. This fluctuation suggests that while the model does exhibit clear convergence, its performance could likely be enhanced further through targeted hyperparameter tuning. Refining these parameters could help stabilize the training process and potentially reduce the final loss even more.

8.3. GRU

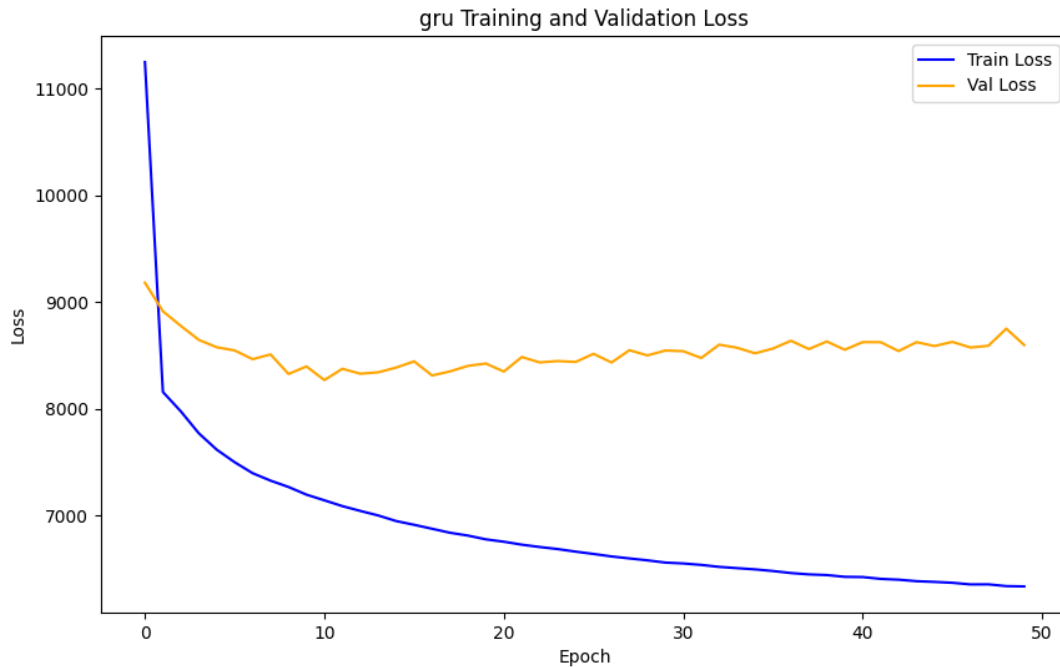


Figure 8.3. Training and Validation Loss of the Best GRU model

The GRU model demonstrates a steady and consistent decline in training loss over the span of 50 epochs, which indicates that the model is successfully minimizing error on the training dataset. However, the validation loss tells a different story, while it initially decreases alongside the training loss during the first 20 epochs, it begins to plateau and eventually diverge slightly as training progresses. This pattern suggests that the model starts to overfit after a certain point, fitting the training data too closely at the expense of its ability to generalize to unseen data.

To address this overfitting, we implemented a strategy where the model's weights were updated and the best-performing checkpoint was saved after each epoch, ensuring that we could retain the version of the model that offered the lowest validation loss. Additionally, techniques such as introducing dropout layers or adjusting regularization parameters could further help mitigate overfitting and improve the model's ability to generalize.

Another notable observation is the difference in smoothness between the training and validation loss curves. The training loss decreases in a stable and predictable manner, but the validation loss exhibits more fluctuation across epochs. This behavior is often indicative of a model that is sensitive to variations in the validation data, possibly due to a smaller validation set, noisy samples, or insufficient regularization. Smoother loss curves generally point to more stable and reliable learning, so the observed inconsistency here suggests that the model's learning process could still be fine-tuned for better stability.

8.4. LSTM

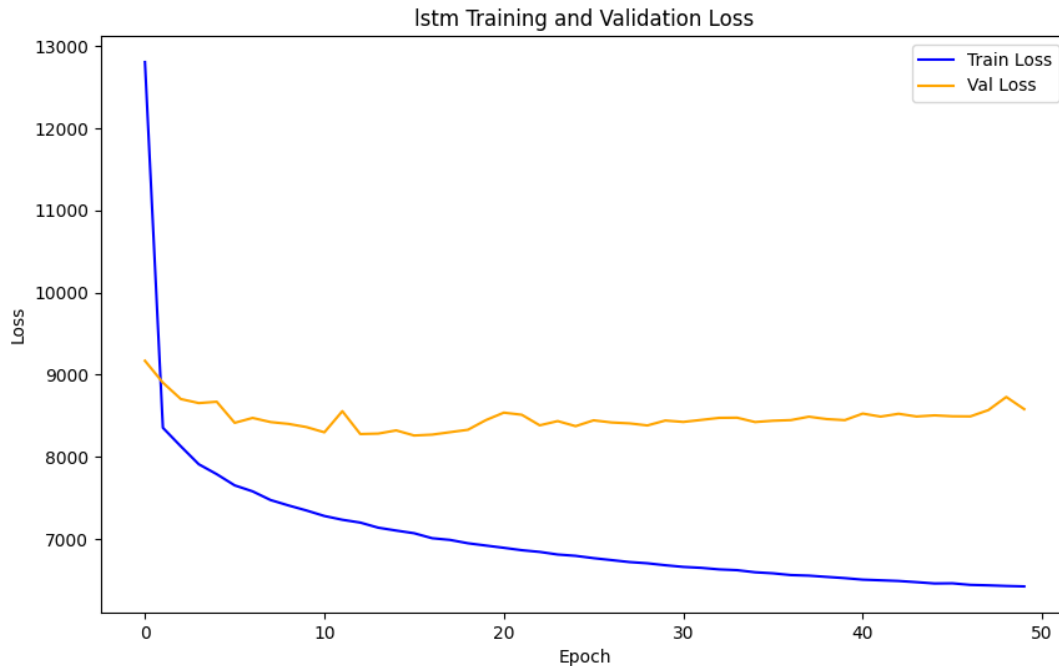


Figure 8.4. Training and Validation Loss of the Best LSTM model

Similar to the GRU model, the LSTM model exhibits a clear and steady downward trend in both its training and validation loss curves over the course of 50 epochs. The close alignment between these two curves suggests that the model generalizes well to unseen data and is not overfitting excessively to the training set. This near-overlapping behavior reflects a balanced learning process where the model successfully minimizes loss on both the training and validation datasets in parallel.

Another encouraging observation is the absence of any major spikes or abrupt plateaus in the loss curves, which indicates that the model's optimization process remained stable throughout training. The validation loss ultimately converges around the 8000 mark, notably the lowest value achieved by any of the models we developed during this experiment, highlighting the LSTM's strong predictive capability for this specific task.

While the training loss graph is smooth and consistent, the validation loss curve does display a few minor spikes in the earlier stages of training. However, these fluctuations gradually diminish as training progresses, and the curve settles into a more stable trajectory toward the end. This behavior is expected, especially when training deep learning models on real-world data, where some level of noise and variability in validation performance is normal. The eventual smooth convergence suggests that the hyperparameter tuning applied to the LSTM was effective in striking a good

balance between learning rate, batch size, and model complexity, ultimately enabling the model to learn robust and generalizable patterns from the data.

8.5. Encoder-Decoder

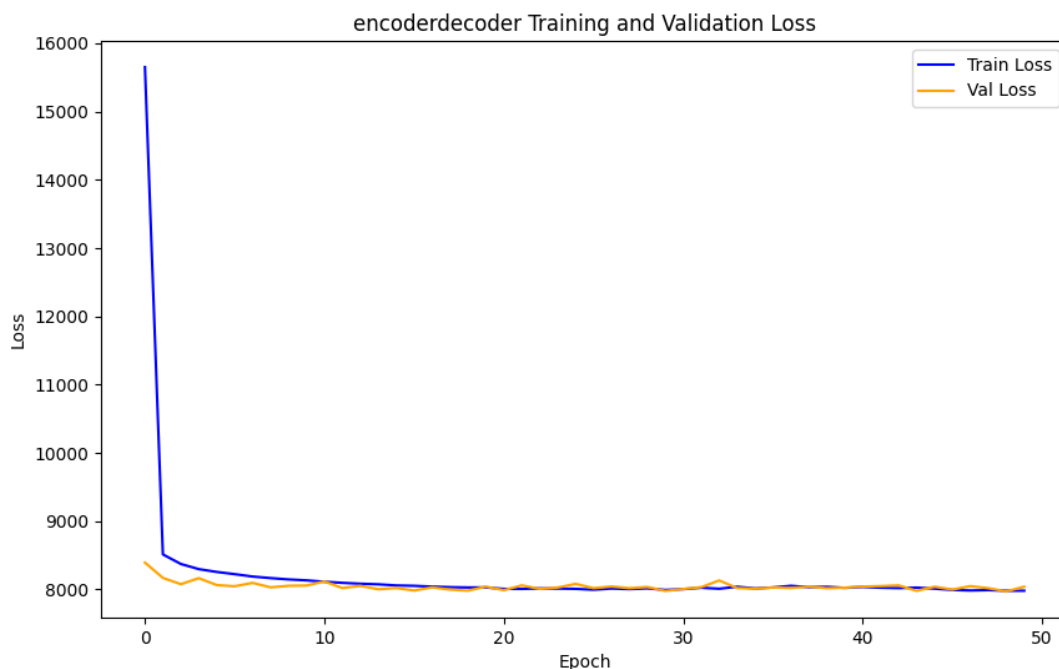


Figure 8.5. Training and Validation Loss of the Best LSTM Encoder-Decoder model

Figure 8.5 shows that the model faced significant difficulty during the training process. Both the training and validation loss curves appear to remain stagnant, hovering around the 8000 mark throughout the entire duration of training. Unlike other models, where a steady decline in loss is observed as the epochs progress, this flat and unchanging trajectory strongly suggests that the model was unable to effectively learn from the data.

Such behavior can often point to a few underlying issues. One likely explanation is that the model became trapped in a local minimum early in the optimization process, preventing it from escaping to a better solution despite further training. Alternatively, the problem could stem from an unsuitable initialization of hyperparameters, such as a learning rate that is too low, an inappropriate weight initialization scheme, or even suboptimal batch sizes, all of which could prevent the model from making meaningful progress during training.

In its current state, the model shows signs of underfitting, as it fails to reduce both training and validation loss over time. To address this, further rounds of hyperparameter tuning would be necessary, potentially experimenting with different optimizers, adjusting the learning rate, modifying regularization techniques, or rethinking the architecture altogether. These steps would

help ensure the model has the capacity and flexibility to capture the underlying patterns in the data more effectively and avoid stagnation in future training runs.

8.6. Transformer

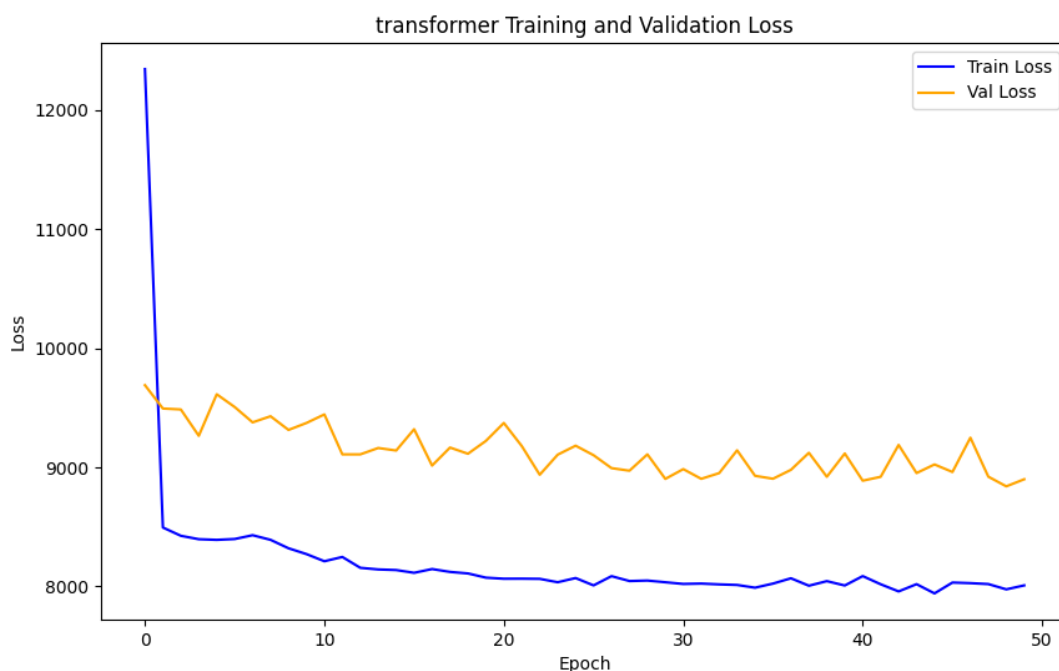


Figure 8.6. Training and Validation Loss of the Best Transformer model

The training and validation loss curves for the Transformer model demonstrate a promising overall trend, with both losses consistently declining across the epochs. This downward trajectory indicates that the model is indeed learning and gradually improving its ability to fit the data. Both the training and validation curves appear to converge toward stable values, suggesting that the model possesses the potential to generalize well.

However, a closer look at the validation loss reveals an important observation, the curve is highly volatile and marked by frequent spikes throughout the training process. While the model does manage to reduce the loss on average, these sharp fluctuations indicate that its learning path is unstable. This type of behavior is often associated with an overly aggressive learning rate, which causes the model to make large, sometimes imprecise, updates to its weights, resulting in inconsistent validation performance from epoch to epoch.

This observation suggests that although the architecture is capable of fitting the data, the current set of hyperparameters, particularly the learning rate, may not be optimally chosen. Tuning the learning rate, along with experimenting with other parameters such as batch size, weight decay,

and learning rate scheduling, could lead to smoother convergence and more stable training. By refining these settings, the Transformer model’s performance and generalization capability could likely be improved significantly, helping reduce the volatility seen in the validation loss and resulting in a more reliable final model.

9. User Manual

9.1. Running the Code

1. Clone the repository from: <https://github.com/ZhiZhangT/FareCaster.git>
2. Navigate to the folder created: `cd FareCaster`
3. Install required dependencies by running: `pip install -r requirements.txt`
4. Create a new sub-directory called `data` in the main directory.
5. Download the data from Kaggle: <https://www.kaggle.com/datasets/dilwong/flightprices>
6. Move `itineraries.csv` into the `data` folder
7. Define the hyperparameters to use for training in the `train.py` script, and run the script to begin training: `python train.py`
8. To access the script used for our grid search hyperparameter tuning experiments: checkout to the “gridsearch” branch in the repository and define the range of hyperparameters to test for in the `train.py` script, before running it.

9.2. Codebase Architecture

The codebase follows a modular design where:

- Data is processed using preprocessing modules
- Models are defined as PyTorch modules in a dedicated file
- Training orchestration is handled by a separate script
- Each experimental run is saved with timestamps for reproducibility and comparison
- Constants and configurations are centralized

9.2.1. Data Preprocessing Pipeline

preprocess.py attempts to either get existing preprocessed data stored in the form of pickle files or find *itineraries.csv* in the `data` folder, prepare it for experimenting, and automatically save it into pickle files within the `saved` folder (Which is automatically created upon running either *preprocess.py* or *train.py*)

preprocess_sota.py achieves the same purpose but includes the additional manipulation of data in order to fit the expected input data shapes and format by the SOTA models imported from the external *neuralforecast* library.

9.2.2. Training Pipeline

train.py is a script that runs training and evaluation for experiments. The script logs information about training such as training plots and time taken for each epoch. It also creates a new folder within `saved` to save the experiment data into. Within this script you can change the hyperparameters for the models and experiments in the `if __name__ == "__main__"` script section. Additionally, the plot / experiment data out settings can also be changed in the same section.

train_sota.py is a script that contains the training and evaluation code for experimenting with SOTA models which interfaces with the *neuralforecast* library used to import these models.

9.2.3. Model Definition

models.py defines the following neural network architectures for time series forecasting:

- Basic Models: GRUModel, LSTMModel - Simple recurrent models
- Deep Models: GRUModelDeep, LSTMModelDeep - Multi-layer versions with dropout
- Advanced Models: LSTMEncoderDecoder, TransformerModel - More sophisticated architectures

Each model follows a consistent interface for compatibility with the training pipeline, which are all using the PyTorch framework.

9.2.4. Constants.py

constants.py centralizes project-wide settings:

- File paths and directories
- Target and feature column definitions
- Categorical and cyclical feature specifications
- Random seeds and other constants

9.2.5. Results and Analyses

The following Python notebooks contain our initial exploration with the dataset and with model training:

- *data_processing.ipynb*
- *EDA.ipynb*
- *transformer.ipynb*

The weights of the best models for each model architecture, including the SOTA models, can be found in the *best_models_weights* folder. The loss plots of these models can be found in the *best_models_losses* folder. The overall metrics for each of the models are stored in the *best_models_comparison.json* file.

Overall results from each set of hyperparameter tuning grid search experiments can be found in the *hyperparam_results* folder. There are 3 Python notebooks in the main directory containing the analyses of results:

- *hyperparam_results_lstm_gru_analysis.ipynb*
- *hyperparam_results_encoderdecoder_analysis.ipynb*
- *hyperparam_results_transformers_analysis.ipynb*

The *saved* folder contains some of the experiment results that were not run on the GPU cluster. Results from the grid search hyperparameter tuning experiments were not pushed to GitHub due to the large amounts of files to track.

10. References

- Calzone, O. (2022, April 10). *An Intuitive Explanation of LSTM*. Medium.
<https://medium.com/@ottaviocalzone/an-intuitive-explanation-of-lstm-a035eb6ab42c>
- Dive Deep into Deep Learning. (n.d.). 11.7. *The Transformer Architecture — Dive into Deep Learning 1.0.0-beta0 documentation*. D2l.ai. https://d2l.ai/chapter_attention-mechanisms-and-transformers/transformer.html
- DProgrammer Org. (2019, April 6). *RNN, LSTM & GRU*. DProgrammer Lopez.
<http://dprogrammer.org/rnn-lstm-gru>
- Jacoby, D., Jonatan Ostrometzky, & Messer, H. (2021). *Short-Term Prediction of the Attenuation in a Commercial Microwave Link Using LSTM-based RNN*.
<https://doi.org/10.23919/eusipco47968.2020.9287835>
- Li, P., Luo, A., Liu, J., Wang, Y., Zhu, J., Deng, Y., & Zhang, J. (2020). Bidirectional Gated Recurrent Unit Neural Network for Chinese Address Element Segmentation. *ISPRS International Journal of Geo-Information*, 9(11), 635. <https://doi.org/10.3390/ijgi9110635>

- Wong, D. (2022). *Flight Prices*. [Www.kaggle.com](https://www.kaggle.com/datasets/dilwong/flightprices).
<https://www.kaggle.com/datasets/dilwong/flightprices>
- Wu, H., Hu, T., Liu, Y., Zhou, H., Wang, J., & Long, M. (2023). TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. *ArXiv:2210.02186 [Cs]*.
<https://arxiv.org/abs/2210.02186>
- Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2023). Are Transformers Effective for Time Series Forecasting? *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(9), 11121–11128. <https://doi.org/10.1609/aaai.v37i9.26317>